



Documentación #1

Lidar: Implementación de Hardware Real y Simulado en ROS 2

MarsRover UDG Space
Generación 2026

Código: 215511699

Alumno: Marquez Pinto Martin Alejandro

Carrera: Ingeniería Robótica (INRO)

Resumen: Integración progresiva de un LiDAR LD19 real y simulado dentro de ROS 2
Jazzy, RViz y Gazebo Harmonic, utilizando una misma interfaz `/scan`.

Guadalajara, Jalisco, México

Resumen Detallado

El objetivo de este proceso fue construir progresivamente una arquitectura de ROS 2 capaz de utilizar un sensor LiDAR LD19 físico y, posteriormente, sustituirlo por un sensor LiDAR simulado en Gazebo sin modificar la lógica principal de procesamiento desarrollada en C++.

El punto central de la arquitectura es el tópico estándar de ROS 2:

`/scan`

El LiDAR físico publica directamente este tópico mediante el driver oficial de LDROBOT. Por otro lado, el LiDAR simulado genera inicialmente un tópico propio del sistema de transporte de Gazebo llamado:

`/scangz`

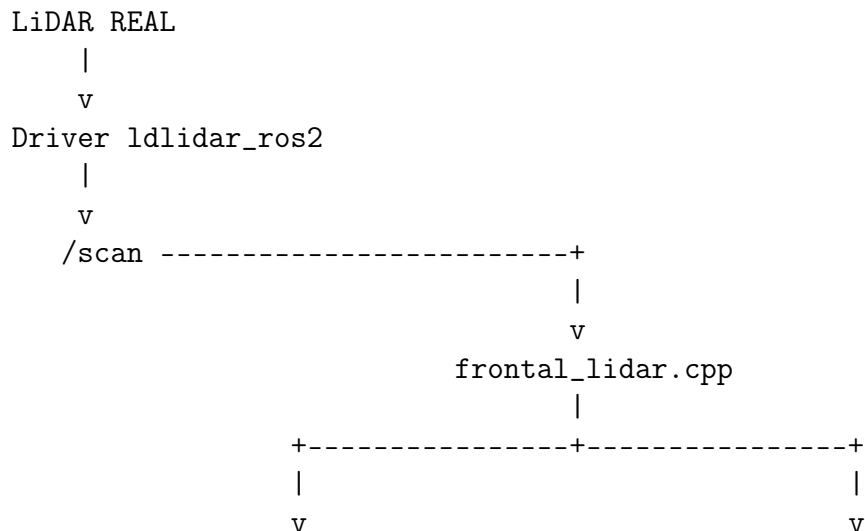
Para comunicar ambos ecosistemas se integró `ros_gz_bridge`, convirtiendo el mensaje:

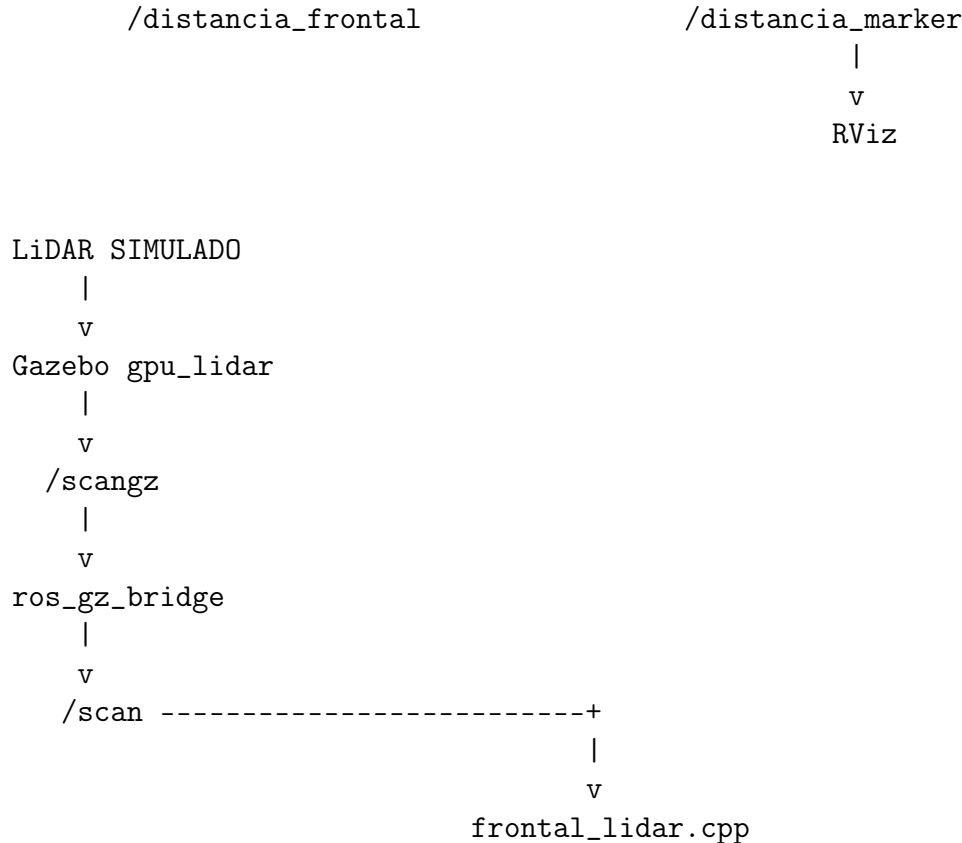
`gz.msgs.LaserScan` $\longrightarrow$ `sensor_msgs/msg/LaserScan`

y publicándolo finalmente como `/scan` dentro de ROS 2.

Gracias a esto, el nodo C++ `frontal_lidar`, originalmente creado para procesar el LD19 real, puede recibir exactamente la misma interfaz cuando el sensor proviene de Gazebo.

La arquitectura conseguida es:





Además del funcionamiento final, durante el proceso se estudiaron conceptos fundamentales de ROS 2, C++, TF, URDF, Xacro, SDF, Gazebo Transport, archivos `package.xml`, instalación de recursos mediante `setup.py` y `CMakeLists.txt`, archivos launch, configuración de RViz y separación entre paquetes de descripción, lógica y bringup.

Una parte especialmente importante del desarrollo fue la depuración. El sistema no funcionó completamente desde el primer intento. Se encontraron problemas relacionados con dependencias, transformaciones TF duplicadas, ausencia de propiedades físicas en el URDF, ubicación incorrecta de la extensión `<gazebo>`, configuración de sensores dentro del mundo, diferencias entre Gazebo Transport y ROS 2, simulación pausada y advertencias de SDF. Estos errores se conservaron en esta documentación porque permitieron entender mejor la arquitectura completa.

Índice general

Resumen Detallado	1
1. Introducción	9
2. Organización del Workspace	10
2.1. rover_description	11
2.2. prueba_cplus	11
2.3. prueba_bringup	11
3. Integración Inicial del LD19 Físico	13
3.1. Sensor utilizado	13
3.2. Workspace independiente para el driver	13
3.3. Submódulos Git	14
3.4. Detección del dispositivo	14
3.5. Problema de compilación del SDK	14
3.6. Cargar el workspace	15
3.7. Ejecución del driver	15
4. Comprensión de LaserScan	16
4.1. Comprobación del tópico	16
4.2. Interpretación de un mensaje	16
4.3. Sistema angular	17
4.4. Sector frontal	17
4.5. Valores infinitos y NaN	18
5. Nodo C++ frontal_lidar	19
5.1. Objetivo	19
5.2. Dependencias del ejecutable	20
5.3. Tres nombres diferentes	20
5.4. Includes	21
5.5. Clase	21
5.6. Constructor	21
5.7. Subscriber	21

5.8. Publishers	22
6. Interfaces de Mensajes Utilizadas	23
6.1. LaserScan	23
6.2. Float32MultiArray	23
6.3. Marker	24
6.4. Point	24
7. Procesamiento del Sector Frontal	25
7.1. Reducción de procesamiento	25
7.2. Recorrido del LaserScan	25
7.3. Filtro frontal	26
7.4. Validación	26
7.5. Conversión a centímetros	26
7.6. Distancia mínima y máxima	26
8. Conversión de Coordenadas Polares a Cartesianas	27
9. Marker para RViz	29
10. Conceptos de C++ Aprendidos Durante el Nodo	31
10.1. Clase	31
10.2. Objeto e instancia	31
10.3. Constructor	31
10.4. Método	31
10.5. Callback	32
10.6. Punteros compartidos	32
10.7. Operador -	32
10.8. Operador	32
10.9. Operador ::	32
10.10 this	33
10.11 std::bind	33
10.12 rclcpp::spin	33
10.13 Nodo, tópico y mensaje	33
11. Integración del Rover en RViz	34
11.1. display.launch.py	34
11.2. robot_state_publisher	34
11.3. joint_state_publisher_gui	35
11.4. RViz	36
12. Transformaciones TF y base_laser	37
13. Construcción del URDF/Xacro	38
13.1. base_footprint	38
13.2. base_link	38

13.3. Visual, collision e inertial	39
13.4. base_joint	39
14.Ruedas del Rover	40
14.1. Problema de doble rotación	40
15.Integración de base_laser	42
15.1. base_laser_joint	42
16.Link auxiliar frontal_laser	44
17.Instalación de Recursos de rover_description	45
17.1. get_package_share_directory	46
18.Dependencias de rover_description	47
18.1. xacro	47
18.2. ros_gz_sim	47
18.3. robot_state_publisher	48
18.4. ros_gz_bridge	48
18.5. joint_state_publisher_gui	48
18.6. rviz2	48
18.7. prueba_bringup	48
19.Creación de gazebo.launch.py	49
19.1. Imports	49
19.2. Ruta de rover_description	49
19.3. Procesamiento del Xacro	50
20.Creación del Mundo rover_world.sdf	51
20.1. Decisión inicial	51
20.2. Estado actual	51
20.3. Comentarios heredados	52
21.Iniciar Gazebo desde ROS 2	53
21.1. Ruta del SDF	53
21.2. Distinción importante	54
21.2.1. Ruta del archivo	54
21.2.2. Nombre interno del mundo	54
22.Spawn del Rover en Gazebo	55
23.Problema Inicial: Rover Visual pero no Físico	57
24.Creación del LiDAR Simulado	58
24.1. Plugin del mundo	58
24.2. Sensor	58
24.3. Parámetros seleccionados	60

25. Error Importante: Ubicación de la Extensión Gazebo	61
25.1. Configuración problemática	61
25.2. Investigación	61
25.3. Causa real	62
25.4. Lección	62
26. Validación del Sensor Dentro de Gazebo	63
27. Gazebo Transport y ROS 2 no son el Mismo Sistema	64
28. Prueba Directa de /scangz	66
28.1. Ranges infinitos	66
28.2. Prueba con una caja	67
28.3. Gazebo en pausa	67
29. vertical_angle_step = NaN	68
30. Warning de gz_frame_id	69
31. Creación del Bridge Gazebo-ROS	70
31.1. Tipos de mensaje	70
31.2. Dirección	70
32. gazebo_bridge.yaml	72
32.1. ros_topic_name	72
32.2. gz_topic_name	72
32.3. ros_type_name	72
32.4. gz_type_name	72
32.5. direction	73
33. Instalación del YAML en prueba_bringup	74
34. Localización del Bridge desde gazebo.launch.py	75
35. Dependencias Entre Paquetes	77
36. Comparación con la Arquitectura de Maya	78
36.1. Comparación	79
36.2. XML, YAML y Python	79
37. Pruebas Experimentales de Bringup	80
38. Bridge Funcionando	81
39. Prueba Definitiva: Reutilizar frontal_lidar	83
39.1. Hardware	83
39.2. Simulación	83

40. Validación Final en RViz	85
41. Bitácora de Errores, Investigación y Soluciones	86
41.1. El driver LD19 no compilaba	86
41.2. RViz no era una prueba suficiente del sensor	86
41.3. TF duplicada	86
41.4. Se sospechó incorrectamente de base_footprint	87
41.5. Faltaban propiedades físicas	87
41.6. Comentarios XML incorrectos	87
41.7. Confusión entre nombre del mundo y archivo SDF	87
41.8. El sensor no aparecía	88
41.9. No aparecía /scan en ROS 2	88
41.10 Ranges infinitos	88
41.11 El tópico parecía detenido	88
41.12 Warning de gz_frame_id	88
41.13 Lección general de depuración	89
42. Línea de Tiempo del Desarrollo	90
43. Estado Actual	93
44. Pendientes y Mejoras Futuras	94
44.1. Revisar gz_frame_id	94
44.2. base_laser_joint	94
44.3. Crear un mundo desde cero	94
44.4. Arquitectura de bringup	94
44.5. ros2_control	94
44.6. Movimiento del rover	95
44.7. Procesamiento LiDAR	95
44.8. Robustez de frontal_lidar	95
A. Comandos Útiles	96
A.1. Compilar workspace	96
A.2. Lanzar RViz	96
A.3. Lanzar Gazebo	96
A.4. Comprobar tópicos ROS	96
A.5. Comprobar tópicos Gazebo	96
A.6. Escuchar scan de Gazebo	97
A.7. Escuchar scan ROS	97
A.8. Ejecutar filtro frontal	97
A.9. Comprobar Marker	97
A.10. Comprobar sensores del modelo	97
B. Glosario	98
C. Arquitectura Final Resumida	100

Capítulo 1

Introducción

El desarrollo comenzó trabajando directamente con un sensor LiDAR LDROBOT LD19 físico. El objetivo inicial era aprender a obtener sus mediciones dentro de ROS 2, interpretar el mensaje `LaserScan`, filtrar únicamente la zona frontal del robot y posteriormente visualizar la información procesada dentro de RViz.

Después de conseguir el funcionamiento con hardware real se decidió avanzar hacia una segunda meta: reproducir el mismo comportamiento en simulación utilizando Gazebo Harmonic.

Esto permitió establecer un objetivo de diseño más general:

La lógica de percepción del robot no debería depender directamente de si los datos provienen de un sensor físico o de un sensor simulado.

Para conseguirlo se mantuvo `/scan` como interfaz común entre ambos sistemas.

El proyecto de práctica se utiliza además como entorno de aprendizaje antes de integrar conceptos equivalentes dentro del proyecto Maya del equipo de robótica.

Capítulo 2

Organización del Workspace

El workspace utilizado actualmente es:

/Data/mars_rover/ros2_ws

La estructura conceptual relevante es aproximadamente:

```
ros2_ws/
|
+-- src/
|   |
|   +-- rover_description/
|   |   |
|   |   +-- launch/
|   |   |   +-- display.launch.py
|   |   |   +-- gazebo.launch.py
|   |   |
|   |   +-- urdf/
|   |   |   +-- rover.urdf.xacro
|   |   |
|   |   +-- rviz2/
|   |   |   +-- rover.rviz
|   |   |
|   |   +-- world/
|   |   |   +-- rover_world.sdf
|   |   |
|   |   +-- package.xml
|   |   +-- setup.py
|   |
+-- prueba_bringup/
|   |
|   +-- config/
|   |   +-- gazebo_bridge.yaml
```

```

|   |
|   +-- launch/
|   +-- CMakeLists.txt
|   +-- package.xml
|
+-- prueba_cplus/
|   |
|   +-- src/
|   |   +-- frontal_lidar.cpp
|   |   +-- ...
|   |
|   +-- CMakeLists.txt
|   +-- package.xml
|
+-- prueba_python/

```

Cada paquete tiene una responsabilidad diferente.

2.1. rover_description

Contiene principalmente la descripción del rover y los recursos necesarios para visualizarlo y simularlo:

- URDF/Xacro.
- Launch de RViz.
- Launch de Gazebo.
- Archivo de configuración de RViz.
- Mundo SDF.

2.2. prueba_cplus

Contiene los nodos escritos en C++. Entre ellos se encuentra `frontal_lidar.cpp`, responsable de procesar `/scan`.

2.3. prueba_bringup

Actualmente almacena, entre otros recursos experimentales, la configuración YAML del bridge entre Gazebo y ROS 2.

Su función conceptual comienza a parecerse a un paquete de *bringup*: un lugar donde se concentran configuraciones y elementos necesarios para levantar varios subsistemas.

Observación arquitectónica

Actualmente `gazebo.launch.py` vive dentro de `rover_description` y busca el archivo YAML localizado en `prueba_bringup`. Por ello, `rover_description` depende actualmente de `prueba_bringup`.

A futuro podría estudiarse una reorganización más parecida a Maya, donde el paquete de `bringup` sea quien coordine al paquete de descripción. No es necesario cambiarlo todavía; se conserva como posible mejora arquitectónica.

Capítulo 3

Integración Inicial del LD19 Físico

3.1. Sensor utilizado

Se utilizó un sensor:

- Fabricante: LDROBOT.
- Modelo: LD19.
- Alcance especificado: aproximadamente 0,02 a 12 m.
- Frecuencia de escaneo típica: 10 Hz.
- Resolución angular típica a 10 Hz: aproximadamente $0,8^\circ$.
- Resolución de medición indicada en datasheet: 15 mm.
- Comunicación serie.
- Baudrate utilizado: 230400.

El datasheet LD19 V2.5 especifica además una frecuencia de ranging interna de aproximadamente 4500 Hz, desviación estándar de aproximadamente 10 mm entre 0,3 y 12 m y una precisión promedio especificada de aproximadamente ± 45 mm bajo las condiciones descritas por el fabricante.

3.2. Workspace independiente para el driver

Se creó un workspace separado para evitar mezclar directamente drivers externos con los paquetes propios del rover:

```
mkdir -p ~/ldlidar_ros2_ws/src
cd ~/ldlidar_ros2_ws/src
```

Posteriormente se clonó el repositorio oficial:

```
git clone https://github.com/ldrobotSensorTeam/ldlidar_ros2.git
```

3.3. Submódulos Git

El repositorio contiene un submódulo denominado `sdk`. Por ello fue necesario:

```
cd ~/ldlidar_ros2_ws/src/ldlidar_ros2
git submodule update --init --recursive
```

Este paso permitió entender una diferencia importante:

Un paquete ROS 2 y un submódulo Git son conceptos diferentes.

Un paquete ROS organiza software para ROS. Un submódulo Git enlaza otro repositorio Git a una revisión específica.

3.4. Detección del dispositivo

El adaptador USB-UART utilizado fue detectado como un dispositivo CP210x.

Se realizaron comprobaciones como:

```
lsusb
ls /dev/ttyUSB*
ls -l /dev/ttyUSB0
```

En el sistema utilizado el dispositivo apareció normalmente como:

`/dev/ttyUSB0`

El usuario ya pertenecía al grupo `dialout`, por lo que no fue necesario utilizar permanentemente permisos inseguros como `chmod 777`.

3.5. Problema de compilación del SDK

Durante la compilación con ROS 2 Jazzy aparecieron errores relacionados con funciones `pthread`:

```
pthread_mutex_init was not declared
pthread_mutex_lock was not declared
pthread_mutex_unlock was not declared
```

El error provenía de:

`sdk/src/log_module.cpp`

La solución aplicada fue añadir:

```
#include <pthread.h>
```

Después se limpió la compilación:

```
cd ~/ldlidar_ros2_ws  
rm -rf build install log  
colcon build
```

La compilación terminó correctamente.

3.6. Cargar el workspace

Después de compilar:

```
source ~/ldlidar_ros2_ws/install/setup.bash
```

Es importante distinguir:

`source` no enciende el LiDAR. Únicamente carga el entorno del workspace y permite que ROS 2 encuentre sus paquetes.

3.7. Ejecución del driver

El sensor se inició mediante:

```
ros2 launch ldlidar_ros2 ld19.launch.py
```

El launch utilizado configuraba parámetros semejantes a:

```
product_name: LDLiDAR_LD19  
laser_scan_topic_name: scan  
point_cloud_2d_topic_name: pointcloud2d  
frame_id: base_laser  
port_name: /dev/ttyUSB0  
serial_baudrate: 230400  
range_min: 0.02  
range_max: 12.0
```

Además, el launch original publicaba una transformación estática:

```
base_link --> base_laser
```

con aproximadamente 0,18 m de elevación.

Capítulo 4

Comprensión de LaserScan

4.1. Comprobación del tópico

Después de iniciar el driver:

```
ros2 topic info /scan
```

se obtuvo un tópico de tipo:

```
sensor_msgs/msg/LaserScan
```

Posteriormente:

```
ros2 topic echo /scan --once
```

permitió observar campos como:

```
header
angle_min
angle_max
angle_increment
time_increment
scan_time
range_min
range_max
ranges
intensities
```

4.2. Interpretación de un mensaje

De manera aproximada, un mensaje `LaserScan` representa un barrido completo del sensor.

Para cada índice i :

$$\theta_i = \text{angle_min} + i \cdot \text{angle_increment} \quad (4.1)$$

y:

$$r_i = \text{ranges}[i] \quad (4.2)$$

donde:

- θ_i es el ángulo correspondiente al rayo.
- r_i es la distancia medida.

4.3. Sistema angular

En el driver utilizado:

- 0° corresponde aproximadamente al frente del sensor.
- El ángulo aumenta en sentido antihorario.
- 90° apunta hacia $+Y$.
- 180° apunta hacia atrás.
- 270° equivale aproximadamente a -90° .

Esto sigue la convención de mano derecha utilizada por ROS.

4.4. Sector frontal

Se decidió estudiar inicialmente un sector de aproximadamente:

$$\pm 45^\circ \quad (4.3)$$

alrededor del frente.

Sin embargo, debido a que el frente coincide con la discontinuidad entre 360° y 0° , el intervalo debe representarse mediante dos regiones:

$$315^\circ \leq \theta < 360^\circ \quad (4.4)$$

o:

$$0^\circ \leq \theta \leq 45^\circ \quad (4.5)$$

En radianes:

$$5,4978 \leq \theta \quad \vee \quad \theta \leq 0,7854 \quad (4.6)$$

Este detalle se volvió especialmente importante posteriormente en Gazebo, donde una caja colocada exactamente enfrente del rover produjo lecturas tanto al inicio como al final del arreglo `ranges`.

No eran dos objetos diferentes: era el mismo obstáculo cruzando la frontera $360^\circ/0^\circ$.

4.5. Valores infinitos y NaN

Una medición puede no representar una distancia válida.

Durante las pruebas aparecieron:

`inf`
`nan`

Estos valores no implican necesariamente una falla del LiDAR.

- `inf`: puede indicar que no existe retorno dentro del rango máximo.
- `nan`: medición no válida o no definida.

Por ello posteriormente se utilizó:

```
std::isfinite(distancia_m)
```

además de comprobar los límites `range_min` y `range_max`.

Capítulo 5

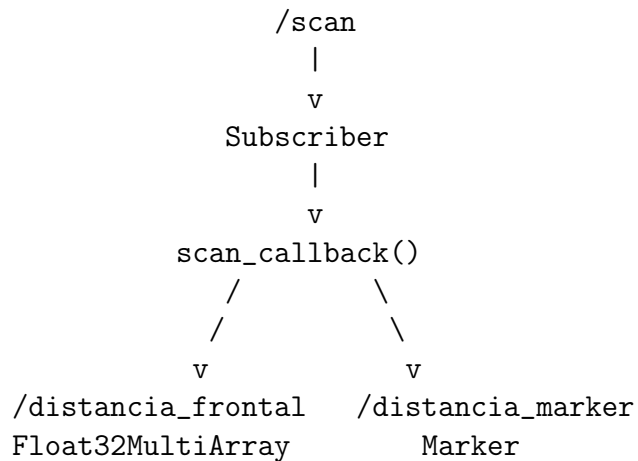
Nodo C++ `frontal_lidar`

5.1. Objetivo

Se creó un nodo propio para:

1. Suscribirse a `/scan`.
2. Analizar únicamente el sector frontal.
3. Validar las mediciones.
4. Convertirlas de metros a centímetros.
5. Publicar un arreglo de distancias.
6. Convertir las coordenadas polares a cartesianas.
7. Crear un Marker para RViz.

La estructura lógica es:



5.2. Dependencias del ejecutable

El paquete `prueba_cplus` utiliza:

```
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(example_interfaces REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(std_msgs REQUIRED)
find_package(visualization_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(prueba_custom_interfaces REQUIRED)
```

Para `frontal_lidar` específicamente:

```
add_executable(frontal_lidar src/frontal_lidar.cpp)

ament_target_dependencies(
  frontal_lidar
  rclcpp
  sensor_msgs
  std_msgs
  visualization_msgs
  geometry_msgs
)
```

Finalmente el ejecutable es instalado para que pueda localizarlo:

```
install(TARGETS
  ...
  frontal_lidar
  ...
  DESTINATION lib/${PROJECT_NAME}
)
```

5.3. Tres nombres diferentes

Un detalle importante aprendido es que pueden existir varios nombres relacionados con un mismo nodo:

- Ejecutable CMake: `frontal_lidar`.
- Clase C++: `frontal_lidar`.
- Nombre del nodo en runtime: `rangefront_lidarcpp`.

Estos conceptos no son obligatoriamente el mismo nombre.

5.4. Includes

El nodo actual utiliza:

```
#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/laser_scan.hpp"
#include "std_msgs/msg/float32_multi_array.hpp"
#include "visualization_msgs/msg/marker.hpp"
#include "geometry_msgs/msg/point.hpp"
#include <cmath>
```

5.5. Clase

La clase principal es:

```
class frontal_lidar : public rclcpp::Node
{
    ...
};
```

Esto significa que `frontal_lidar` hereda las capacidades proporcionadas por:

`rclcpp::Node`

5.6. Constructor

El constructor contiene:

```
frontal_lidar() : Node("rangefront_lidarcpp")
```

Al crearse un objeto de esta clase también se inicializa el nodo de ROS 2.

5.7. Subscriber

El subscriber se crea mediante:

```
subscriber_ =
    this->create_subscription<sensor_msgs::msg::LaserScan>(
        "/scan",
        10,
        std::bind(
            &frontal_lidar::scan_callback,
            this,
            std::placeholders::_1
        )
    );
```

El nodo escucha:

/scan

y espera mensajes:

sensor_msgs/msg/LaserScan

5.8. Publishers

El primer publisher utiliza:

```
publisher_distancias =  
    this->create_publisher<std_msgs::msg::Float32MultiArray>(  
        "/distancia_frontal",  
        10  
    );
```

El segundo utiliza:

```
publisher_maker =  
    this->create_publisher<visualization_msgs::msg::Marker>(  
        "/distancia_marker",  
        10  
    );
```

Por lo tanto:

```
/scan  
|  
+--> frontal_lidar  
    |  
    +--> /distancia_frontal  
    |  
    +--> /distancia_marker
```

Capítulo 6

Interfaces de Mensajes Utilizadas

Durante el desarrollo surgió una confusión importante con el término *custom interface*.

Las interfaces utilizadas por `frontal_lidar`:

- `sensor_msgs/msg/LaserScan`
- `std_msgs/msg/Float32MultiArray`
- `visualization_msgs/msg/Marker`
- `geometry_msgs/msg/Point`

no son interfaces personalizadas. Son mensajes estándar ya definidos por ROS 2.

Una interfaz verdaderamente personalizada sería, por ejemplo:

`prueba_custom_interfaces/msg/EstadoHardware.msg`

El paquete `prueba_cplus` sí contiene una dependencia a `prueba_custom_interfaces`, pero actualmente es utilizada por otros nodos del paquete y no por `frontal_lidar`.

6.1. LaserScan

Representa un conjunto de mediciones angulares de distancia.

Es la entrada principal del nodo.

6.2. Float32MultiArray

Se utilizó para publicar un arreglo de distancias del sector frontal:

`/distancia_frontal`

Cada valor se almacena mediante:

```
mensaje.data.push_back(distancia_cm);
```

Para mediciones inválidas actualmente se inserta:

```
mensaje.data.push_back(NAN);
```

6.3. Marker

El Marker se utiliza únicamente para visualización.

El Marker actual es del tipo:

```
visualization_msgs::msg::Marker::POINTS
```

equivalente al valor numérico:

type: 8

6.4. Point

Cada punto del Marker utiliza:

```
geometry_msgs::msg::Point
```

con componentes:

x

y

z

Capítulo 7

Procesamiento del Sector Frontal

7.1. Reducción de procesamiento

El nodo contiene actualmente:

```
contador++;

if (contador <= 3)
{
    return;
}

contador = 0;
```

Por lo tanto, en la versión actual se procesan aproximadamente:

1 de cada 4 mensajes recibidos.

Esto es diferente de reducir la frecuencia del LiDAR.

El sensor sigue publicando a su frecuencia configurada; simplemente nuestro callback decide ignorar temporalmente algunos mensajes.

7.2. Recorrido del LaserScan

El arreglo se recorre mediante:

```
for (size_t i = 0; i < msg->ranges.size(); ++i)
```

El ángulo correspondiente se calcula:

```
float angulo =
    msg->angle_min +
    i * msg->angle_increment;
```

7.3. Filtro frontal

La condición actual es:

```
if (angulo >= 5.4978f || angulo <= 0.7854f)
```

correspondiente aproximadamente a:

315° -> 360°

OR

0° -> 45°

7.4. Validación

Posteriormente:

```
if (!std::isfinite(distancia_m) ||
    distancia_m < msg->range_min ||
    distancia_m > msg->range_max)
{
    mensaje.data.push_back(NAN);
    continue;
}
```

Esto evita usar mediciones fuera del rango físico definido por el mensaje.

7.5. Conversión a centímetros

```
float distancia_cm = distancia_m * 100.0f;
```

7.6. Distancia mínima y máxima

Actualmente se actualizan:

```
dist_min = std::min(distancia_cm, dist_min);
dist_max = std::max(distancia_cm, dist_max);
```

Estas variables se utilizan principalmente para información de depuración en terminal.

Capítulo 8

Conversión de Coordenadas Polares a Cartesianas

El LiDAR entrega una distancia y un ángulo:

$$(r, \theta) \tag{8.1}$$

pero RViz necesita coordenadas espaciales.

Se utiliza:

$$x = r \cos(\theta) \tag{8.2}$$

$$y = r \sin(\theta) \tag{8.3}$$

En código:

```
coordenadas.x =  
    distancia_m * std::cos(angulo);  
  
coordenadas.y =  
    distancia_m * std::sin(angulo);  
  
coordenadas.z = 0.0;
```

Después:

```
points.points.push_back(coordenadas);
```

Durante la prueba final con una caja en Gazebo se observaron puntos semejantes a:

$x = 0.584\dots$
 $y = 0.000\dots$

$x = 0.584\dots$
 $y = 0.008\dots$

$x = 0.584\dots$
 $y = 0.016\dots$

El hecho de que x permaneciera aproximadamente constante mientras y cambiaba mostró que los puntos estaban reconstruyendo la superficie casi plana del obstáculo.

Capítulo 9

Marker para RViz

El Marker actual se configura aproximadamente mediante:

```
visualization_msgs::msg::Marker points;  
  
points.type =  
    visualization_msgs::msg::Marker::POINTS;  
  
points.header.frame_id = "base_laser";  
points.header.stamp = msg->header.stamp;  
  
points.action =  
    visualization_msgs::msg::Marker::ADD;  
  
points.scale.x = 0.05;  
points.scale.y = 0.05;  
  
points.color.r = 0.0;  
points.color.g = 0.0;  
points.color.b = 0.0;  
points.color.a = 1.0;  
  
points.ns = "frontal_lidar";  
points.id = 100;
```

Posteriormente se publica:

```
publisher_maker->publish(points);
```

El tópico resultante es:

/distancia_marker

Durante la prueba final:

```
frame_id: base_laser  
ns: frontal_lidar  
id: 100  
type: 8
```

confirmó que el nodo estaba produciendo correctamente un Marker de tipo POINTS.

Capítulo 10

Conceptos de C++ Aprendidos Durante el Nodo

10.1. Clase

Una clase define la estructura y comportamiento de un tipo de objeto.

Ejemplo:

```
class frontal_lidar : public rclcpp::Node
```

10.2. Objeto e instancia

Cuando se ejecuta:

```
auto node =  
    std::make_shared<frontal_lidar>();
```

se crea una instancia real de la clase.

10.3. Constructor

Es una función especial ejecutada cuando se crea el objeto.

10.4. Método

Una función perteneciente a una clase.

Por ejemplo:

```
scan_callback()
```

10.5. Callback

Un callback es una función que no se ejecuta necesariamente de forma secuencial por nuestra propia llamada directa. ROS 2 la ejecuta cuando ocurre un evento, en este caso cuando llega un nuevo mensaje de `/scan`.

10.6. Punteros compartidos

ROS 2 utiliza frecuentemente:

`SharedPtr`
`ConstSharedPtr`

Estos representan punteros inteligentes con administración automática de memoria.

10.7. Operador -

Cuando se tiene un puntero:

```
msg->ranges
```

equivale conceptualmente a acceder a un miembro del objeto apuntado.

10.8. Operador .

Cuando se trabaja directamente con un objeto:

```
points.scale.x
```

se utiliza el operador punto.

10.9. Operador ::

Indica resolución de ámbito.

Por ejemplo:

```
sensor_msgs::msg::LaserScan
```

puede leerse aproximadamente como:

`LaserScan`
dentro de `msg`
dentro de `sensor_msgs`

10.10. `this`

Dentro de una clase:

```
this->create_publisher(...)
```

`this` representa al objeto actual.

10.11. `std::bind`

`std::bind` permite asociar el callback con la instancia actual de la clase.

10.12. `rclcpp::spin`

En:

```
rclcpp::spin(node);
```

ROS mantiene el nodo activo y atiende eventos como mensajes entrantes.

10.13. Nodo, tópico y mensaje

Una analogía útil utilizada durante el aprendizaje fue:

Nodo	= programa / componente
Topico	= cable virtual
Publisher	= quien escribe en el cable
Subscriber	= quien escucha el cable
Mensaje	= formato de los datos

Capítulo 11

Integración del Rover en RViz

11.1. display.launch.py

Se creó un launch específico para visualizar el modelo.

Primero se localiza el paquete:

```
pkg_path =  
    get_package_share_directory(  
        'rover_description'  
    )
```

Luego se construye la ruta:

```
xacro_file = os.path.join(  
    pkg_path,  
    'urdf',  
    'rover.urdf.xacro'  
)
```

Posteriormente se procesa:

```
robot_description_raw =  
    xacro.process_file(  
        xacro_file  
    ).toxml()
```

11.2. robot_state_publisher

El launch ejecuta:

```
Node(  
    package='robot_state_publisher',
```

```

    executable='robot_state_publisher',
    output='screen',
    parameters=[
        {
            'robot_description':
            robot_description_raw
        }
    ]
)

```

Su función no es únicamente “mostrar el robot en RViz”.

Su función principal es utilizar la descripción cinemática del robot para publicar las transformaciones TF correspondientes.

Conceptualmente:

```

URDF/Xacro
  |
  v
robot_description
  |
  +---- joint_states
  |
  v
robot_state_publisher
  |
  v
TF
  |
  +----> RViz
  +----> otros nodos ROS

```

11.3. joint_state_publisher_gui

También se lanza:

```

Node(
    package='joint_state_publisher_gui',
    executable='joint_state_publisher_gui',
    output='screen'
)

```

Este nodo permite generar estados de articulaciones durante pruebas cuando todavía no existe un controlador físico o `ros2_control` publicándolos.

11.4. RViz

Finalmente:

```
Node(  
    package='rviz2',  
    executable='rviz2',  
    arguments=[  
        "-d",  
        os.path.join(  
            pkg_path,  
            'rviz2',  
            'rover.rviz'  
        )  
    ],  
    name='rviz2',  
    output='screen'  
)
```

El archivo:

`rviz2/rover.rviz`

permite conservar configuraciones como:

- Fixed Frame.
- RobotModel.
- TF.
- LaserScan.
- Marker.
- Tópicos seleccionados.
- Opciones visuales.

Capítulo 12

Transformaciones TF y base_laser

Inicialmente el launch oficial del LD19 publicaba una transformación estática:

```
base_link --> base_laser
```

Posteriormente se integró `base_laser` directamente al URDF del rover.

Esto produjo una situación importante:

Dos nodos diferentes podían intentar convertirse en autoridad de la misma transformación TF.

La solución aplicada fue dejar al URDF junto con `robot_state_publisher` como autoridad de la transformación.

De esta forma la transformación del LiDAR forma parte de la estructura general del robot.

Capítulo 13

Construcción del URDF/Xacro

13.1. base_footprint

Se utiliza:

```
<link name="base_footprint"/>
```

Una observación importante fue que este link puede estar vacío.

Inicialmente se sospechó que Gazebo fallaba porque `base_footprint` no tenía:

- visual,
- collision,
- inertial.

Sin embargo, posteriormente se comprobó que esta conclusión era incorrecta.

`base_footprint` funciona correctamente como frame auxiliar sin geometría física.

El problema real comenzaba en los links que sí debían comportarse como cuerpos físicos.

13.2. base_link

El cuerpo principal utiliza:

```
<box size="0.4 0.3 0.1"/>
```

equivalente aproximadamente a:

- Largo: 0,40 m.
- Ancho: 0,30 m.
- Alto: 0,10 m.

El link incluye:

- `visual`
- `collision`
- `inertial`

La masa utilizada actualmente es:

$$m = 1 \text{ kg} \quad (13.1)$$

con inercias:

`Ixx = 0.00833`

`Iyy = 0.01416`

`Izz = 0.02083`

13.3. Visual, collision e inertial

Se aprendió a distinguir:

visual: determina cómo se observa el objeto.

collision: determina el volumen utilizado por el motor físico.

inertial: define masa, centro de masa y tensor de inercia.

Un modelo puede verse correctamente en RViz pero seguir siendo físicamente incompleto para Gazebo.

13.4. base_joint

```
<joint name="base_joint" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0 0 0.10"
    rpy="0 0 0"/>
</joint>
```

El chasis queda elevado aproximadamente 10 cm respecto a `base_footprint`.

Capítulo 14

Ruedas del Rover

Para evitar repetir cuatro veces el mismo código se creó un macro Xacro:

```
<xacro:macro
  name="wheel"
  params="prefix x_reflect y_reflect">
```

Cada rueda utiliza:

- Radio: 0,06 m.
- Longitud: 0,04 m.
- Masa: 0,3 kg.

Las inercias actuales son aproximadamente:

$I_{xx} = 0.00031$

$I_{yy} = 0.00031$

$I_{zz} = 0.00054$

Las posiciones son generadas con:

```
<origin
  xyz="{x_reflect * 0.12}
    {y_reflect * 0.17}
    -0.04"
  rpy="1.5708 0 0"/>
```

14.1. Problema de doble rotación

Durante el desarrollo se analizó la orientación de los cilindros.

La articulación ya contiene:

$rpy = "1.5708 \ 0 \ 0"$

por lo que no era necesario aplicar nuevamente la misma rotación dentro del `visual`, `collision` e `inertial`.

De lo contrario se produciría una doble transformación.

Capítulo 15

Integración de base_laser

El LiDAR se representa mediante un cilindro:

```
<cylinder
  radius="0.025"
  length="0.05"/>
```

con masa:

$$m = 0,2 \text{ kg} \quad (15.1)$$

e inercias:

$I_{xx} = 0.00007292$

$I_{yy} = 0.00007292$

$I_{zz} = 0.0000625$

Su posición respecto a `base_link` es:

```
<origin
  xyz="0.175 0 0.075"
  rpy="0 0 0"/>
```

15.1. base_laser_joint

Actualmente se conserva:

```
<joint
  name="base_laser_joint"
  type="continuous">
```

Esto fue útil durante el proceso de aprendizaje para visualizar la articulación.

Nota importante

Un LiDAR LD19 instalado físicamente sobre el rover normalmente debería representarse como un elemento fijo respecto al chasis:

`type="fixed"`

El cuerpo exterior del LiDAR no gira alrededor del rover; el mecanismo de barrido se encuentra dentro del propio sensor.

Por el momento se conserva `continuous` deliberadamente, pero se marca como una posible corrección futura.

Capítulo 16

Link auxiliar frontal_laser

Se creó además:

`frontal_laser`

como una geometría visual auxiliar.

No representa un segundo sensor.

Su propósito es ayudar a identificar visualmente la orientación del LiDAR.

El link se conecta mediante:

```
<joint
  name="frontal_laser_joint"
  type="fixed">
```

a `base_laser`.

Capítulo 17

Instalación de Recursos de rover_description

El paquete utiliza `ament_python`.

En `setup.py` se agregaron:

```
(os.path.join(
    'share',
    package_name,
    'launch'
), glob('launch/*.launch.py')),

(os.path.join(
    'share',
    package_name,
    'urdf'
), glob('urdf/*.xacro')),

(os.path.join(
    'share',
    package_name,
    'rviz2'
), glob('rviz2/*.rviz')),

(os.path.join(
    'share',
    package_name,
    'world'
), glob('world/*.sdf')),
```

Esto permite que, después de:

```
colcon build
```

```
source install/setup.bash
```

los archivos estén disponibles dentro del espacio de instalación.

Conceptualmente:

```
src/rover_description/...
    |
    | colcon build
    v
install/rover_description/
  share/rover_description/
    launch/
    urdf/
    rviz2/
    world/
```

17.1. `get_package_share_directory`

Posteriormente:

```
get_package_share_directory(
    'rover_description'
)
```

localiza precisamente el directorio `share` de la instalación del paquete.

No busca arbitrariamente la carpeta dentro de `src`.

Capítulo 18

Dependencias de rover_description

El bloque actualizado de dependencias de ejecución es:

```
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>
<exec_depend>ament_index_python</exec_depend>
<exec_depend>xacro</exec_depend>
<exec_depend>ros_gz_sim</exec_depend>
<exec_depend>robot_state_publisher</exec_depend>
<exec_depend>ros_gz_bridge</exec_depend>
<exec_depend>joint_state_publisher_gui</exec_depend>
<exec_depend>rviz2</exec_depend>

<exec_depend>prueba_bringup</exec_depend>
```

18.1. xacro

`xacro` sí debe considerarse una dependencia necesaria porque los launches ejecutan:

```
import xacro
```

y:

```
xacro.process_file(...)
```

18.2. ros_gz_sim

Es necesario para lanzar Gazebo mediante su launch y utilizar el ejecutable `create` para realizar el spawn.

18.3. `robot_state_publisher`

Es utilizado directamente por `display.launch.py`.

18.4. `ros_gz_bridge`

Es ejecutado directamente desde `gazebo.launch.py` mediante:
`parameter_bridge`

18.5. `joint_state_publisher_gui`

Es utilizado en el launch de RViz.

18.6. `rviz2`

Es ejecutado directamente por `display.launch.py`.

18.7. `prueba_bringup`

Es necesario actualmente porque `gazebo.launch.py` busca:
`prueba_bringup/config/gazebo_bridge.yaml`

Pendiente menor

Una versión anterior del `package.xml` contenía además:

```
<depend>rclpy</depend>
```

Los launches mostrados actualmente no utilizan directamente `rclpy`. Puede revisarse posteriormente si esta dependencia sigue siendo necesaria por algún otro archivo del paquete.

Capítulo 19

Creación de gazebo.launch.py

El launch de Gazebo se creó desde cero con tres responsabilidades principales:

1. Iniciar Gazebo.
2. Insertar el rover.
3. Iniciar el bridge Gazebo-ROS.

19.1. Imports

```
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources \
    import PythonLaunchDescriptionSource

from ament_index_python.packages \
    import get_package_share_directory

from launch_ros.actions import Node

import os
import xacro
```

19.2. Ruta de rover_description

```
pkg_path =
    get_package_share_directory(
        'rover_description'
    )
```

19.3. Procesamiento del Xacro

```
urdf_file = os.path.join(
    pkg_path,
    'urdf',
    'rover.urdf.xacro'
)

rover_description_urdf =
    xacro.process_file(
        urdf_file
    ).toxml()
```

Esto convierte el Xacro a una cadena URDF que posteriormente puede recibir Gazebo.

Capítulo 20

Creación del Mundo rover_world.sdf

Para comenzar no se diseñó un mundo completamente nuevo.

Se tomó como base el mundo:

`empty.sdf`

incluido con Gazebo Harmonic.

20.1. Decisión inicial

El contenido de `empty.sdf` fue copiado al paquete propio y guardado como:

`world/rover_world.sdf`

El nombre interno se cambió a:

```
<world name="rover_world">
```

20.2. Estado actual

La configuración original de `empty.sdf` se conservó casi completamente de forma intencional.

De momento solamente se añadió el sistema necesario para sensores:

```
<plugin  
  filename="gz-sim-sensors-system"  
  name="gz::sim::systems::Sensors">  
  <render_engine>ogre2</render_engine>  
</plugin>
```

El mundo conserva además elementos heredados como:

- Physics.

- UserCommands.
- SceneBroadcaster.
- Contact.
- Luz tipo sun.
- Ground plane.

20.3. Comentarios heredados

En la cabecera del archivo todavía existen ejemplos heredados de `empty.sdf` que hacen referencia a rutas como:

```
/world/empty/create  
/world/empty/remove
```

Estos comentarios no corresponden al nombre actual `rover_world`.

No afectan la simulación porque están comentados, pero se conservan como evidencia de que el archivo actual fue obtenido directamente del ejemplo original.

Plan futuro

El mundo actual se considera una solución inicial para aprender el proceso de integración.

Más adelante se planea construir un mundo desde cero y definir explícitamente sus propiedades, modelos, terreno, iluminación y plugins necesarios.

Capítulo 21

Iniciar Gazebo desde ROS 2

Se obtiene la ruta del paquete:

```
direccion_gz =  
    get_package_share_directory(  
        "ros_gz_sim"  
    )
```

Después se incluye el launch oficial:

```
gazebo =  
    IncludeLaunchDescription(  
        PythonLaunchDescriptionSource(  
            os.path.join(  
                direccion_gz,  
                "launch",  
                "gz_sim.launch.py"  
            )  
        ),  
        launch_arguments={  
            "gz_args": sdf_file  
        }.items()  
    )
```

21.1. Ruta del SDF

Previamente:

```
sdf_file = os.path.join(  
    pkg_path,  
    'world',  
    'rover_world.sdf'  
)
```

21.2. Distinción importante

Durante el proceso se confundieron dos conceptos diferentes:

21.2.1. Ruta del archivo

```
../../rover_world.sdf
```

Esta ruta se utiliza para decirle a Gazebo qué archivo cargar.

21.2.2. Nombre interno del mundo

```
<world name="rover_world">
```

Este nombre identifica al mundo una vez cargado.

No debe confundirse con la ruta del archivo.

Capítulo 22

Spawn del Rover en Gazebo

El rover se inserta mediante:

```
rover_spawn = Node(  
    package="ros_gz_sim",  
    executable="create",  
    output="screen",  
    parameters=[  
        {  
            'robot_description':  
                rover_description_urdf  
        }  
    ],  
    arguments=[  
        "-string", rover_description_urdf,  
        "-name", "Bochito",  
        "-world", "rover_world",  
        "-x", "0",  
        "-y", "0",  
        "-z", "0"  
    ]  
)
```

El flujo conceptual es:

```
rover.urdf.xacro  
    |  
    v  
xacro.process_file()  
    |  
    v  
Cadena URDF  
    |  
    v
```

```
ros_gz_sim create
|
v
Conversión / importación en Gazebo
|
v
Bochito
```

Capítulo 23

Problema Inicial: Rover Visual pero no Físico

Durante las primeras pruebas el modelo podía visualizarse, pero Gazebo no conseguía interpretar correctamente todos sus elementos físicos.

Inicialmente se sospechó de:

`base_footprint`

por estar vacío.

Posteriormente se comprobó que esto no era el problema.

El verdadero problema era que varios links físicos no poseían correctamente:

- collision,
- inertial,
- masa,
- tensor de inercia.

Estos elementos fueron añadidos progresivamente al:

- chasis,
- ruedas,
- cuerpo del LiDAR.

Esto permitió que el rover pudiera pasar de ser principalmente una descripción visual para RViz a un modelo mucho más adecuado para simulación física.

Capítulo 24

Creación del LiDAR Simulado

La guía oficial de sensores de Gazebo Harmonic se utilizó como referencia para declarar el sensor.

24.1. Plugin del mundo

Primero se añadió al mundo:

```
<plugin
  filename="gz-sim-sensors-system"
  name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>
```

Este plugin pertenece al:

```
<world>
```

y no directamente al link del robot.

24.2. Sensor

Posteriormente se añadió al Xacro:

```
<gazebo reference="base_laser">

  <sensor
    name="ld19"
    type="gpu_lidar">

    <pose>
      0 0 0 0 0 0
    </pose>
```

```

<always_on>true</always_on>
<visualize>true</visualize>

<topic>scangz</topic>

<update_rate>10</update_rate>

<gz_frame_id>
  base_laser
</gz_frame_id>

<ray>

  <scan>

    <horizontal>
      <samples>453</samples>
      <min_angle>0</min_angle>
      <max_angle>
         $\{2\pi\}$ 
      </max_angle>
    </horizontal>

    <vertical>
      <samples>1</samples>
    </vertical>

  </scan>

  <range>
    <min>0.02</min>
    <max>12.0</max>
    <resolution>
      0.015
    </resolution>
  </range>

</ray>

</sensor>

</gazebo>

```

24.3. Parámetros seleccionados

update_rate = 10: frecuencia típica del LD19.

min = 0.02: distancia mínima.

max = 12.0: distancia máxima.

resolution = 0.015: resolución lineal aproximada de 15 mm.

vertical samples = 1: LiDAR 2D.

horizontal samples = 453: densidad utilizada para aproximar el comportamiento angular observado.

El paso angular observado posteriormente fue aproximadamente:

$$0,01390085 \text{ rad} \tag{24.1}$$

equivalente aproximadamente a:

$$0,7965^\circ \tag{24.2}$$

muy cercano a la resolución angular típica de aproximadamente $0,8^\circ$ indicada por el datasheet para 10 Hz.

Capítulo 25

Error Importante: Ubicación de la Extensión Gazebo

Este fue uno de los errores que más tiempo tomó encontrar.

25.1. Configuración problemática

Inicialmente la estructura era conceptualmente:

```
<link name="base_laser">

    ...

    <gazebo reference="base_laser">
        <sensor ...>
        </sensor>
    </gazebo>

</link>
```

El sensor no aparecía correctamente en Gazebo.

25.2. Investigación

Se probaron varias hipótesis:

- Falta del plugin Sensors.
- Error del tipo `gpu_lidar`.
- Problemas del frame.
- Parámetros incorrectos.

- Problemas en el mundo.

Ninguna de estas hipótesis explicaba completamente el comportamiento.

25.3. Causa real

La extensión:

```
<gazebo reference="base_laser">
```

estaba colocada dentro del propio:

```
<link name="base_laser">
```

La estructura fue corregida a:

```
<link name="base_laser">
  ...
</link>

<gazebo reference="base_laser">
  <sensor ...>
    ...
  </sensor>
</gazebo>
```

Después de este cambio Gazebo consiguió reconocer el sensor.

25.4. Lección

El atributo:

```
reference="base_laser"
```

ya indica a qué link debe asociarse la extensión.

No era necesario colocar el bloque dentro de la definición del link.

Capítulo 26

Validación del Sensor Dentro de Gazebo

Se utilizó:

```
gz model -m Bochito --sensor
```

Después de corregir la estructura apareció el sensor:

```
Sensor: ld19
Parent: Bochito
Range min: 0.02
Range max: 12
Resolution: 0.015
Horizontal samples: 453
Min angle: 0
Max angle: 6.28319
Vertical samples: 1
```

Esto permitió comprobar que el sensor había sobrevivido correctamente a la cadena:

```
Xacro
|
v
URDF
|
v
Interpretación / conversión de Gazebo
|
v
SDF interno
|
v
gpu_lidar
```

Capítulo 27

Gazebo Transport y ROS 2 no son el Mismo Sistema

Un descubrimiento importante fue que:

`gz topic`

y:

`ros2 topic`

pertenecen a sistemas de comunicación diferentes.

Gazebo utiliza:

Gazebo Transport.

ROS 2 utiliza:

DDS a través de las abstracciones de ROS 2.

Por ello inicialmente podía existir:

`/scangz`

al ejecutar:

```
gz topic -l
```

sin que apareciera ningún:

`/scan`

en:

```
ros2 topic list
```

Esto no era un error del sensor.

Simplemente faltaba comunicar ambos ecosistemas.

Capítulo 28

Prueba Directa de /scangz

Se utilizó:

```
gz topic -e -t /scangz
```

El mensaje de Gazebo mostró campos aproximadamente como:

```
frame: "base_laser"
```

```
angle_max:
```

```
6.2831853071795862
```

```
angle_step:
```

```
0.01390085244951236
```

```
range_min:
```

```
0.02
```

```
range_max:
```

```
12
```

```
count:
```

```
453
```

```
vertical_count:
```

```
1
```

28.1. Ranges infinitos

Inicialmente aparecían muchos:

```
inf
```

Esto generó dudas sobre si el LiDAR estaba funcionando.

Se comprobó posteriormente que significaban:

El rayo no encontró ninguna superficie dentro de los 12 metros configurados.

28.2. Prueba con una caja

Se colocó un obstáculo frente al rover.

Entonces comenzaron a aparecer distancias finitas aproximadamente como:

0.826
0.81
0.78
...
0.73

Los rayos cercanos al borde recorrían una distancia mayor que los rayos cercanos a la perpendicular de la cara de la caja.

Esto confirmó que el sensor estaba midiendo geometría real del mundo simulado.

28.3. Gazebo en pausa

En una prueba parecía que el tópico había dejado de publicar.

La causa resultó ser que la simulación estaba pausada.

Cuando Gazebo está detenido:

```
tiempo simulado detenido
    |
    v
sensor no actualiza
    |
    v
no aparecen nuevos mensajes
```

Al presionar *Play*, el tópico volvió a actualizarse.

Esta prueba permitió entender la diferencia entre tiempo de simulación y tiempo del sistema.

Capítulo 29

`vertical_angle_step = NaN`

El mensaje del LiDAR simulado mostró:

```
vertical_count: 1  
vertical_angle_step: nan
```

Esto no representa necesariamente un error.

El sensor tiene solamente una capa vertical:

```
<vertical>  
  <samples>1</samples>  
</vertical>
```

Por lo tanto no existe una separación angular entre múltiples capas verticales.

Esto es coherente con un LiDAR 2D.

Capítulo 30

Warning de gz_frame_id

Gazebo genera actualmente un warning semejante a:

```
XML Element[gz_frame_id],  
child of element[sensor],  
not defined in SDF.  
Copying[gz_frame_id] as  
children of [sensor].
```

Este warning aparece durante la conversión / interpretación del modelo.

Sin embargo, posteriormente:

```
ros2 topic echo /scan --once
```

produjo:

```
header:  
  stamp:  
    sec: ...  
    nanosec: ...  
  frame_id: base_laser
```

Por lo tanto el warning no está bloqueando actualmente el funcionamiento del sistema.

Estado

Pendiente de revisión.

No se considera resuelto definitivamente. Es posible que posteriormente se estudie una forma más adecuada de definir el frame sin producir la advertencia.

Capítulo 31

Creación del Bridge Gazebo–ROS

Después de confirmar que:

`/scangz`

estaba funcionando dentro de Gazebo, fue necesario llevar la información hacia ROS 2.

Se utilizó:

`ros_gz_bridge`

La documentación oficial de Gazebo confirma que este paquete permite comunicar mensajes entre ROS 2 y Gazebo Transport.

31.1. Tipos de mensaje

Del lado de Gazebo:

`gz.msgs.LaserScan`

Del lado ROS:

`sensor_msgs/msg/LaserScan`

31.2. Dirección

Para este sensor únicamente necesitamos:

Gazebo --> ROS

porque Gazebo produce la medición.

Esto se representa como:

`GZ_TO_ROS`

Otros tópicos futuros, por ejemplo comandos de motores, pueden requerir:

ROS_TO_GZ

Por lo tanto la dirección del bridge debe estudiarse por tópico y no necesariamente como una configuración global para todo el robot.

Capítulo 32

gazebo__bridge.yaml

Se creó:

prueba_bringup/config/gazebo_bridge.yaml

con:

```
- ros_topic_name: "/scan"
  gz_topic_name: "/scangz"
  ros_type_name: "sensor_msgs/msg/LaserScan"
  gz_type_name: "gz.msgs.LaserScan"
  direction: GZ_TO_ROS
```

32.1. ros__topic__name

Nombre que tendrá el tópico dentro de ROS 2:

/scan

32.2. gz__topic__name

Nombre original dentro de Gazebo Transport:

/scangz

32.3. ros__type__name

sensor_msgs/msg/LaserScan

32.4. gz__type__name

gz.msgs.LaserScan

32.5. direction

GZ_TO_ROS

Capítulo 33

Instalación del YAML en prueba_bringup

prueba_bringup utiliza:

ament_cmake

El CMake actual contiene:

```
install(DIRECTORY
  launch
  config
  DESTINATION
    share/${PROJECT_NAME}/
)
```

Por lo tanto:

```
src/prueba_bringup/config/
    |
    | colcon build
    v
install/prueba_bringup/
  share/prueba_bringup/config/
```

Esto permite localizar posteriormente:

gazebo_bridge.yaml

utilizando las herramientas estándar de ROS 2.

Capítulo 34

Localización del Bridge desde gazebo.launch.py

Primero:

```
direccion_bringup =  
    get_package_share_directory(  
        'prueba_bringup'  
    )
```

Después:

```
gazebo_bridge =  
    os.path.join(  
        direccion_bringup,  
        'config',  
        'gazebo_bridge.yaml'  
    )
```

Finalmente:

```
gz_to_ros_bridge = Node(  
    package="ros_gz_bridge",  
    executable="parameter_bridge",  
    parameters=[  
        {  
            "config_file":  
                gazebo_bridge  
        }  
    ]  
)
```

Y el launch termina devolviendo:

```
return LaunchDescription([
    gazebo,
    rover_spawn,
    gz_to_ros_bridge
])
```

Capítulo 35

Dependencias Entre Paquetes

Durante esta etapa se consolidó una regla útil:

El paquete A depende de B cuando A necesita recursos, nodos, configuraciones o ejecutables de B para funcionar.

Actualmente:

```
rover_description
|
v
prueba_bringup
```

porque `gazebo.launch.py` localizado en `rover_description` busca el YAML de `prueba_bringup`.

Por ello:

```
<exec_depend>
  prueba_bringup
</exec_depend>
```

tiene sentido dentro de `rover_description/package.xml`.

Por otra parte, el paquete `prueba_bringup` contiene actualmente:

```
<exec_depend>prueba_python</exec_depend>
<exec_depend>prueba_cplus</exec_depend>
```

porque sus launches experimentales pueden iniciar nodos pertenecientes a esos paquetes.

Capítulo 36

Comparación con la Arquitectura de Maya

Durante el desarrollo se inspeccionó el proyecto Maya como referencia real de una arquitectura más grande.

A primera vista se observó una separación similar a:

```
maya_description
maya_bringup
maya_aruco
...
```

El launch principal estudiado está escrito en XML y funciona como coordinador de varios subsistemas.

Dentro de él se encontró una estructura equivalente a:

```
<let
  name="gazebo_config_path"
  value="$(find-pkg-share maya_bringup)/
    config/gazebo_bridge.yaml" />
```

y posteriormente:

```
<node
  pkg="ros_gz_bridge"
  exec="parameter_bridge">

  <param
    name="config_file"
    value="$(var gazebo_config_path)" />

</node>
```

36.1. Comparación

Maya utiliza conceptualmente:

```
Launch general XML
    |
    v
maya_bringup/config/
    |
    v
gazebo_bridge.yaml
```

Nuestro proyecto actualmente utiliza:

```
display.launch.py
gazebo.launch.py
    |
    v
prueba_bringup/config/
    |
    v
gazebo_bridge.yaml
```

La sintaxis es diferente, pero el principio arquitectónico es similar:

La configuración del bridge puede mantenerse en un YAML separado mientras un launch se encarga de localizarlo e iniciar el proceso.

36.2. XML, YAML y Python

La documentación de `ros_gz_bridge` estudiada mostró varias posibilidades:

- Ejecutar `parameter_bridge` manualmente.
- Describir el mapeo de tópicos en YAML.
- Lanzar el bridge desde un launch XML.
- Lanzar el bridge desde un launch Python.
- Utilizar sintaxis de launch YAML en los casos soportados.

Esto permitió concluir que la elección entre XML y Python para el launch es una decisión de organización y estilo; no significa que el bridge requiera obligatoriamente un formato específico.

Capítulo 37

Pruebas Experimentales de Bringup

Dentro de `prueba_bringup` existen launches utilizados exclusivamente para aprender la sintaxis de launch de ROS 2.

Estos archivos no se consideran actualmente parte del flujo principal del rover.

Sin embargo, de ellos se obtuvieron conceptos que pueden ser útiles posteriormente:

- Lanzar nodos C++ y Python desde un mismo launch.
- Escribir launches equivalentes en XML y Python.
- Incluir un launch desde otro launch.
- Remapear nombres de nodos.
- Remapear tópicos.
- Remapear servicios.
- Utilizar namespaces.

Estos conceptos pueden resultar especialmente importantes cuando Maya requiera organizar múltiples sensores, controladores y nodos dentro de una sola arquitectura.

Capítulo 38

Bridge Funcionando

Después de ejecutar el sistema apareció:

Creating GZ->ROS Bridge:

```
[/scangz  
  (gz.msgs.LaserScan)  
  ->  
  /scan  
  (sensor_msgs/msg/LaserScan)]
```

Al ejecutar:

```
gz topic -l
```

aparecía:

```
/scangz
```

Mientras que:

```
ros2 topic list
```

mostró:

```
/scan
```

sin necesidad de conectar físicamente el LD19.

Esto confirmó:

```
Gazebo Transport  
  |  
  v  
  /scangz  
  |  
  v
```

```
ros_gz_bridge
|
v
ROS 2 /scan
```

Capítulo 39

Prueba Definitiva: Reutilizar frontal_lidar

El nodo `frontal_lidar` ya estaba programado para escuchar:

`/scan`

de tipo:

`sensor_msgs/msg/LaserScan`

El bridge genera exactamente esa misma interfaz.

Por lo tanto no fue necesario modificar el algoritmo.

39.1. Hardware

```
LD19
|
v
ldlidar_ros2
|
v
/scan
|
v
frontal_lidar
```

39.2. Simulación

```
Gazebo gpu_lidar
|
v
```

```
/scangz
|
v
ros_gz_bridge
|
v
/scan
|
v
frontal_lidar
```

Este resultado representa uno de los avances más importantes del proyecto:

La lógica de procesamiento quedó desacoplada del origen físico o simulado de la medición.

Capítulo 40

Validación Final en RViz

Para mostrar la medición original se utiliza un display:

`LaserScan`

Topic: `/scan`

Para mostrar el filtro:

`Marker`

Topic: `/distancia_marker`

Durante pruebas simples puede utilizarse:

Fixed Frame: `base_laser`

Cuando toda la TF del rover está activa también pueden utilizarse frames superiores como `base_link` o `base_footprint`, siempre que exista la cadena TF correspondiente.

Capítulo 41

Bitácora de Errores, Investigación y Soluciones

Esta sección conserva deliberadamente los problemas encontrados.

41.1. El driver LD19 no compilaba

Síntoma:

Errores relacionados con `pthread`.

Causa encontrada:

Faltaba la inclusión correspondiente dentro del SDK utilizado.

Solución:

```
#include <pthread.h>
```

41.2. RViz no era una prueba suficiente del sensor

Antes de culpar a RViz se decidió comprobar:

```
ros2 topic info /scan
ros2 topic echo /scan --once
```

Esto permitió separar problemas de adquisición de problemas de visualización.

41.3. TF duplicada

Síntoma:

Tanto el launch oficial del driver como el URDF podían publicar `base_link ->base_laser`.

Solución:

Dejar la transformación bajo la autoridad de `robot_state_publisher` mediante el URDF.

41.4. Se sospechó incorrectamente de `base_footprint`

El link vacío parecía inicialmente una posible causa de errores de Gazebo.

Posteriormente se comprobó que puede funcionar correctamente como frame auxiliar.

41.5. Faltaban propiedades físicas

Los links principales necesitaban:

```
collision
inertial
mass
inertia
```

para una representación adecuada dentro del motor físico.

41.6. Comentarios XML incorrectos

En algún punto se utilizó sintaxis semejante a:

```
# comentario
```

dentro del XML.

La sintaxis correcta es:

```
<!-- comentario -->
```

41.7. Confusión entre nombre del mundo y archivo SDF

Se aprendió que:

```
rover_world.sdf
```

es el archivo, mientras:

```
rover_world
```

es el nombre interno definido por:

```
<world name="rover_world">
```

41.8. El sensor no aparecía

Se investigaron plugins, frames y parámetros.

Finalmente la causa principal fue que:

```
<gazebo reference="base_laser">
```

estaba dentro de:

```
<link name="base_laser">
```

Moverlo fuera del link solucionó el problema.

41.9. No aparecía /scan en ROS 2

Situación:

/scangz funcionaba.

Causa:

Gazebo Transport y ROS 2 son sistemas separados.

Solución:

Integrar `ros_gz_bridge`.

41.10. Ranges infinitos

Inicialmente parecían indicar una falla.

La prueba colocando un objeto frente al robot demostró que simplemente no existía retorno dentro del alcance máximo.

41.11. El tópico parecía detenido

La simulación estaba en pausa.

Presionar *Play* permitió que avanzara el reloj de simulación y se generaran nuevos scans.

41.12. Warning de gz_frame_id

No se ha resuelto todavía.

El sistema funciona y el mensaje ROS contiene:

```
frame_id: base_laser
```

pero Gazebo sigue notificando que `gz_frame_id` no forma parte del esquema SDF esperado en esa posición.

Se conserva como tema pendiente.

41.13. Lección general de depuración

El proceso mostró la utilidad de comprobar cada capa por separado:

```
¿Existe el modelo?  
|  
v  
¿Existe el sensor?  
|  
v  
¿Existe /scangz?  
|  
v  
¿Tiene datos /scangz?  
|  
v  
¿Está activo el bridge?  
|  
v  
¿Existe /scan?  
|  
v  
¿Tiene datos /scan?  
|  
v  
¿frontal_lidar publica?  
|  
v  
¿RViz recibe el Marker?
```

Esto evita modificar múltiples capas simultáneamente sin saber dónde se encuentra el problema.

Capítulo 42

Línea de Tiempo del Desarrollo

1. Instalación del driver oficial LD19.
2. Inicialización de su submódulo SDK.
3. Resolución del problema de compilación pthread.
4. Comunicación con el LD19 físico.
5. Obtención de `/scan`.
6. Estudio de `sensor_msgs/msg/LaserScan`.
7. Identificación de la convención angular.
8. Diseño del sector frontal $\pm 45^\circ$.
9. Creación de `frontal_lidar.cpp`.
10. Creación de un subscriber para `/scan`.
11. Creación de `/distancia_frontal`.
12. Uso de `Float32MultiArray`.
13. Validación de `inf` y `nan`.
14. Conversión polar-cartesiana.
15. Creación de `/distancia_marker`.
16. Visualización de puntos mediante Marker.
17. Integración de `base_laser` al URDF.
18. Eliminación de TF duplicada.
19. Creación de `display.launch.py`.
20. Integración de `robot_state_publisher`.

21. Integración de `joint_state_publisher_gui`.
22. Creación de `rover.rviz`.
23. Instalación de recursos mediante `setup.py`.
24. Inicio del trabajo con Gazebo Harmonic.
25. Creación de `gazebo.launch.py`.
26. Uso de `gz_sim.launch.py`.
27. Creación temporal de `rover_world.sdf` a partir de `empty.sdf`.
28. Spawn de Bochito mediante `ros_gz_sim create`.
29. Corrección de collision e inertial.
30. Confirmación de que `base_footprint` puede permanecer vacío.
31. Integración física del chasis.
32. Integración física de las ruedas.
33. Integración física de `base_laser`.
34. Creación de la extensión Gazebo del LiDAR.
35. Incorporación del Sensors system al mundo.
36. Depuración del LiDAR que no aparecía.
37. Descubrimiento del error de anidamiento de `<gazebo>`.
38. Validación mediante `gz model -sensor`.
39. Obtención de `/scangz`.
40. Estudio de Gazebo Transport.
41. Prueba de rangos infinitos.
42. Prueba con caja frontal.
43. Estudio del comportamiento con simulación pausada.
44. Comparación con la arquitectura de Maya.
45. Creación de `prueba_bringup`.
46. Creación de `gazebo_bridge.yaml`.
47. Instalación del YAML mediante CMake.
48. Localización del YAML desde `gazebo.launch.py`.
49. Ejecución de `parameter_bridge`.
50. Conversión `/scangz ->/scan`.

51. Confirmación de `frame_id=base_laser`.
52. Ejecución del mismo nodo `frontal_lidar`.
53. Publicación correcta de `/distancia_marker`.
54. Validación del sistema completo en RViz.

Capítulo 43

Estado Actual

Al terminar esta etapa:

- El modelo Bochito aparece en Gazebo.
- Existe un mundo propio denominado `rover_world`.
- El mundo utiliza temporalmente la configuración base de `empty.sdf`.
- El LiDAR simulado existe dentro de Gazebo.
- El LiDAR publica `/scangz`.
- `/scangz` utiliza `gz.msgs.LaserScan`.
- El bridge convierte la información a ROS 2.
- ROS recibe `/scan`.
- `/scan` utiliza `sensor_msgs/msg/LaserScan`.
- El frame recibido es `base_laser`.
- `frontal_lidar` puede procesar el sensor simulado.
- `/distancia_frontal` se publica.
- `/distancia_marker` se publica.
- RViz puede visualizar la medición original y los puntos procesados.
- La misma lógica C++ puede funcionar con LiDAR físico y simulado.

Capítulo 44

Pendientes y Mejoras Futuras

44.1. Revisar `gz_frame_id`

Investigar una forma SDF plenamente válida de controlar el frame del sensor sin generar la advertencia actual.

44.2. `base_laser_joint`

Evaluar finalmente cambiar:

`continuous`

por:

`fixed`

para representar mejor el montaje físico real.

44.3. Crear un mundo desde cero

Sustituir progresivamente la copia actual de `empty.sdf` por un mundo diseñado explícitamente para el rover.

44.4. Arquitectura de `bringup`

Evaluar mover progresivamente la coordinación general hacia `prueba_bringup`, acercando la organización a la utilizada en Maya.

44.5. `ros2_control`

Integrar posteriormente controladores y actuadores simulados.

44.6. Movimiento del rover

Agregar control de ruedas y comandos de movimiento.

44.7. Procesamiento LiDAR

Continuar con:

1. División del scan en más sectores.
2. Detección básica de obstáculos.
3. Filtrado de ruido.
4. PointCloud.
5. Localización.
6. SLAM.
7. Nav2.

44.8. Robustez de `frontal_lidar`

Puede revisarse posteriormente:

- Protección adicional antes de acceder a `mensaje.data[0]`.
- Diferenciar explícitamente cantidad de rayos del sector y cantidad de mediciones válidas.
- Revisar qué comportamiento se desea cuando todos los rayos son inválidos.
- Parametrizar los límites angulares en lugar de mantener constantes numéricas.
- Permitir cambiar el tópico de entrada mediante parámetros o remapping.

Apéndice A

Comandos Útiles

A.1. Compilar workspace

```
cd /Data/mars_rover/ros2_ws  
colcon build  
source install/setup.bash
```

A.2. Lanzar RViz

```
ros2 launch \  
rover_description \  
display.launch.py
```

A.3. Lanzar Gazebo

```
ros2 launch \  
rover_description \  
gazebo.launch.py
```

A.4. Comprobar tópicos ROS

```
ros2 topic list
```

A.5. Comprobar tópicos Gazebo

```
gz topic -l
```

A.6. Escuchar scan de Gazebo

```
gz topic -e -t /scangz
```

A.7. Escuchar scan ROS

```
ros2 topic echo /scan --once
```

A.8. Ejecutar filtro frontal

```
ros2 run prueba_cplus frontal_lidar
```

A.9. Comprobar Marker

```
ros2 topic echo \  
/distancia_marker --once
```

A.10. Comprobar sensores del modelo

```
gz model -m Bochito --sensor
```

Apéndice B

Glosario

Nodo: Programa o componente individual participante en el sistema ROS 2.

Tópico: Canal de comunicación asíncrono utilizado para intercambiar mensajes.

Publisher: Entidad que publica mensajes en un tópico.

Subscriber: Entidad que recibe mensajes de un tópico.

Mensaje: Estructura de datos utilizada para intercambiar información.

Callback: Función ejecutada en respuesta a un evento, como la llegada de un mensaje.

Clase: Plantilla de C++ que define datos y comportamiento de objetos.

Objeto: Instancia concreta de una clase.

Constructor: Método especial ejecutado cuando se crea un objeto.

Puntero: Variable que almacena o referencia una dirección de memoria.

SharedPtr: Puntero inteligente con administración compartida de su ciclo de vida.

Dependencia: Paquete o biblioteca requerida para compilar o ejecutar otro componente.

Workspace: Directorio que agrupa uno o más paquetes ROS 2.

Paquete: Unidad organizativa básica de software en ROS 2.

URDF: Formato XML utilizado para describir la estructura de un robot.

Xacro: Sistema de macros que permite generar URDF de forma más reutilizable.

SDF: Formato utilizado ampliamente por Gazebo para describir mundos, modelos y sensores.

Link: Cuerpo o frame perteneciente a la estructura del robot.

Joint: Relación cinemática entre dos links.

TF: Sistema de transformaciones espaciales entre frames.

base_footprint: Frame auxiliar utilizado como referencia del robot respecto al suelo.

base_link: Frame principal del cuerpo del robot.

base_laser: Frame asociado al sensor LiDAR.

RViz: Herramienta de visualización de ROS.

Gazebo: Simulador utilizado para física, sensores y entorno virtual.

Gazebo Transport: Sistema propio de mensajería utilizado internamente por Gazebo.

ros_gz_bridge: Paquete encargado de convertir mensajes entre ROS 2 y Gazebo Transport.

Bringup: Paquete o capa encargada normalmente de coordinar el arranque de múltiples componentes de un sistema.

Description package: Paquete que concentra principalmente la descripción geométrica y cinemática del robot.

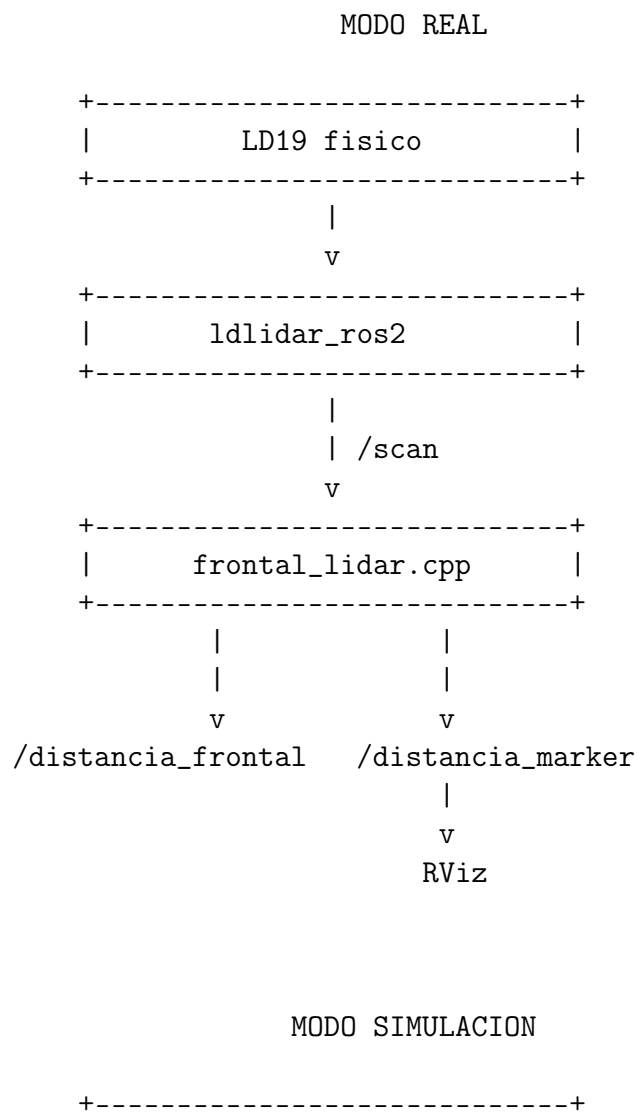
LaserScan: Mensaje que representa un conjunto ordenado de mediciones angulares de distancia.

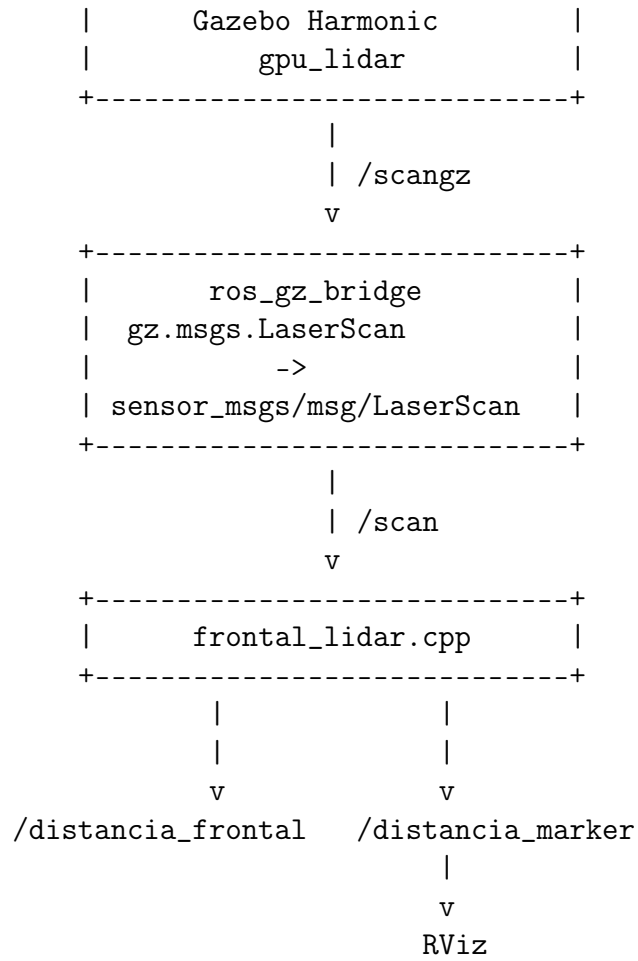
Marker: Mensaje diseñado para enviar elementos de visualización a RViz.

Point: Mensaje que contiene coordenadas cartesianas x , y y z .

Apéndice C

Arquitectura Final Resumida





Referencias e Hipervínculos:

Referencia 1: Gazebo Harmonic – Sensors Tutorial.

[Click Aquí](#)

Referencia 2: Gazebo Harmonic – ROS 2 Integration / ros_gz_bridge.

[Click Aquí](#)

Referencia 3: LDROBOT LD19 Datasheet V2.5.

[Click Aquí](#)

Referencia 4: Repositorio oficial del driver ROS 2 para LDROBOT LiDAR.

[Click Aquí](#)

Archivos propios utilizados como referencia durante esta documentación:

- `rover.urdf.xacro`
- `display.launch.py`
- `gazebo.launch.py`
- `rover_world.sdf`
- `rover.rviz`
- `rover_description/package.xml`
- `rover_description/setup.py`
- `prueba_bringup/config/gazebo_bridge.yaml`
- `prueba_bringup/CMakeLists.txt`
- `prueba_bringup/package.xml`
- `prueba_cplus/src/frontal_lidar.cpp`

- `prueba_cplus/CMakeLists.txt`
- `prueba_cplus/package.xml`
- Nota previa: Incorporar Lidar a RViz.md
- `maya.launch.xml`, utilizado como referencia arquitectónica.
- `gazebo_bridge.yaml`, Referencia del traductor bridge en maya package